

INFORME DE ACTIVIDAD 1: IMPLEMENTACIÓN DE RED DE PROTEÍNAS CON LISTAS ENLAZADAS

Asignatura: Programación, algoritmos y estructuras de datos - Unidad 2

Alumno: Antonio Bellido Maya

1. INTRODUCCIÓN

El objetivo de esta actividad ha sido modelar una red de interacciones proteína-proteína (PPI) utilizando estructuras de datos dinámicas. En concreto, se ha implementado una **lista enlazada** para gestionar las conexiones de cada proteína, sustituyendo el uso de estructuras estáticas como los arrays. Esto permite simular la conectividad biológica y detectar proteínas críticas o "hubs".

2. ANÁLISIS DE RESULTADOS

Tras la ejecución del algoritmo implementado en Python, se ha generado la estructura de la red y se han calculado las conexiones. Los resultados obtenidos son:

- **Estructura de la red:** Se han establecido nodos (proteínas) como P53, ATM y BRCA1. Tras aplicar las inserciones y una eliminación de prueba (MDM2-CDK2), la red mantiene sus enlaces correctamente.
- **Identificación del Hub:** El análisis automático determinó que la proteína **P53** es la más conectada (proteína HUB), con un total de **4 interacciones** activas. Esto concuerda con la biología real, donde P53 actúa como nodo central en la respuesta al daño del ADN.

3. DISCUSIÓN DE EFICIENCIA Y ESTRUCTURAS DE DATOS

La elección de una **Lista Enlazada** frente a un Array (arreglo) presenta ventajas y desventajas justificadas en el contexto de la bioinformática:

1. **Eficiencia en Inserción/Eliminación:** En una red biológica que cambia dinámicamente (descubrimiento de nuevas interacciones), la lista enlazada es superior. La inserción al inicio tiene una complejidad de **O(1)**, ya que solo requerimos actualizar la referencia de la cabeza y del nuevo nodo, sin necesidad de desplazar elementos en memoria como ocurriría en un array estático.
2. **Gestión de Memoria:** A diferencia de los arrays que requieren memoria contigua, la lista enlazada utiliza bloques de memoria dispersos conectados por punteros

(siguiente). Esto ofrece flexibilidad, aunque conlleva un ligero sobrecoste de memoria adicional para almacenar dichas referencias.

3. **Acceso:** La limitación encontrada es el acceso secuencial. Para buscar si una proteína ya interactúa con otra (contiene), debemos recorrer la lista nodo a nodo ($O(n)$), lo cual es menos eficiente que el acceso directo por índice de un array.

4. CONCLUSIONES

La implementación cumple con los requisitos de manipulación dinámica de datos. El uso de clases propias para Nodo y Lista Enlazada ha permitido comprender a bajo nivel cómo se gestionan las referencias en memoria, garantizando la integridad de la red al agregar o eliminar interacciones biológicas.